

Production Change Summary

Last updated: 2026-03-01 (IST)

1) Non-Technical Summary (For Business/Operations)

What was done

- Duplicate product creation is now blocked, so the same stock item cannot be added again by mistake.
- Invoice format has been improved to show HSN and GST rate as separate, clearly visible details.
- Customer registration Step 3 now has improved product matching, including custom model cases, so parts should appear more reliably.
- **Other** compatibility behavior is handled as universal in matching logic.
- Manager role access was updated so manager can approve invoices but cannot delete products or invoices.

What you should check in production (manual user checks)

- Try creating a product with same Name + Brand + Model as an existing product. It should be blocked with duplicate message.
- Generate and download an invoice PDF. Confirm each line shows:
 - Product Name
 - HSN
 - GST %
 - GST Amount
 - Total
- In customer registration Step 3:
 - Check normal brand/model selection.
 - Check custom model via **Other**.
 - Confirm parts list appears correctly and is not empty when compatible products exist.
- Login as Manager:
 - Confirm invoice approve action is available.
 - Confirm product delete is not available.
 - Confirm invoice delete is not available.
- Login as Admin:
 - Confirm existing delete flows still work as before.

Important note

- Changes were implemented without running production DB tests, as requested.
 - Final runtime confirmation depends on production/staging user flow checks.
-

2) Short Technical Summary

- `server/auth.ts`
 - Manager permissions updated:
 - `products` : removed `delete`
 - `invoices` : removed `delete`
 - kept `approve` and `reject`
 - `server/routes.ts`
 - Added normalized duplicate checks for:
 - `POST /api/products`
 - `PATCH /api/products/:id`
 - `POST /api/products/import`
 - Duplicate check supports legacy records (`name`) and current field (`productName`).
 - Passed `gstPercentage` through all invoice PDF generation flows.
 - `server/models/Invoice.ts`
 - Added `gstPercentage` to invoice items schema.
 - `server/utils/generateInvoicePDF.ts`
 - Invoice line-item table updated to separate `HSN` , `GST %` , and `GST Amt` .
 - `client/src/pages/CustomerRegistration.tsx`
 - Compatibility matching made normalized and more resilient.
 - `Other + custom model` path now supports parts section visibility.
 - Universal `* - Other` compatibility included in matching logic.
 - `client/src/pages/Products.tsx`
 - Delete and delete-duplicates UI now hidden when user lacks product delete permission.
 - Compatibility helper text updated to clarify `other` behavior.
-

3) Detailed Technical Summary

A) Duplicate Product Prevention

Implemented in `server/routes.ts` :

- Added helper methods:
 - `normalizeProductField(...)`
 - `escapeRegex(...)`
 - `findExistingProductByIdentity(...)`
- Duplicate key used: `productName + brand + model` (case-insensitive, trimmed).
- Legacy compatibility included:
 - Duplicate lookup checks both `productName` and old `name` fields.
- Applied at:
 - Create route (`POST /api/products`)
 - Update route (`PATCH /api/products/:id` , excluding current record)
 - Import route (`POST /api/products/import` , duplicate rows are skipped and reported as row errors)

Result:

- Duplicate products are blocked at API level.
- Import does not silently create same logical product multiple times.

B) Invoice HSN and GST Rate Separation

Implemented across model, route mapping, and PDF renderer:

- `server/models/Invoice.ts`
 - Added `gstPercentage` in invoice item schema with default `18` .
- `server/routes.ts`
 - All invoice-to-PDF mapping paths now include `gstPercentage` field.
- `server/utils/generateInvoicePDF.ts`
 - Item table columns changed to:
 - `Product Name`
 - `HSN`
 - `Qty`
 - `Unit`
 - `GST %`
 - `GST Amt`
 - `Total`
 - GST rate rendered per line using `item.gstPercentage` fallback to `18` .
 - HSN shown in its own column rather than mixed into name.

Result:

- PDF structure now supports separate HSN/GST visibility per item.

C) Customer Registration Step 3 Product Visibility

Implemented in `client/src/pages/CustomerRegistration.tsx` :

- Added normalized text matching to reduce strict string mismatch issues.
- Compatibility matching now checks:
 - explicit compatibility entries
 - universal tokens
 - brand/model fallback patterns
 - in-stock inventory only
- For model selection:
 - parts section now appears for:
 - normal model selection
 - `Other` model when custom model is entered
- Empty-state message now uses effective selected model label.

Result:

- Step 3 should show parts in more real scenarios, especially where data format is inconsistent.

D) Universal `other` Compatibility

Implemented in matching behavior:

- `normalizedCompat.endsWith(" - other")` is treated as universal compatibility behavior in Step 3 matching.
- Product form helper text now explicitly states that selecting model `Other` is universal.

Result:

- `Other` handling is explicit and consistent at UI guidance + matching layers.

E) Manager Access Rules

Backend enforcement:

- `server/auth.ts` updated role permissions so Manager has:
 - invoice approval/rejection access
 - no delete permission for products
 - no delete permission for invoices

Frontend visibility:

- `client/src/pages/Products.tsx`
 - Delete and delete-duplicates controls wrapped by `canDeleteProducts` .
- `client/src/pages/Invoices.tsx`
 - Approve buttons available for Admin/Manager on pending approval state.
 - Delete remains Admin-only in UI.

API protection:

- `server/routes.ts`
 - delete product and delete invoice routes already guarded by `requirePermission(..., 'delete')` .
 - approve invoice route guarded by `requirePermission('invoices', 'approve')` .

Result:

- Manager can approve invoices but cannot delete products/invoices through normal UI or protected API routes.

Verification and Risk Notes

- Static check (`npm run check`) was executed.
- Existing unrelated project TypeScript issues remain; these were pre-existing and outside this change scope.
- Because no production DB/runtime testing was executed by design, final confidence requires manual production/staging UAT against real data.